

assignment2

Assignment #2: 位运算、前缀和、树状数组、归并排序 & 状态压缩

Updated 2026-03-17 11:00 GMT+8
Compiled by 徐前 物理学院 (2026 Spring)

作业的各项评分细则及对应的得分

标准	等级	得分
按时提交	完全按时提交：1分 提交有请假说明：0.5分 未提交：0分	1 分
源码、耗时（可选）、解题思路（可选）	提交了4个或更多题目且包含所有必要信息：1分 提交了2个或以上题目但不足4个：0.5分 少于2个：0分	1 分
AC代码截图	提交了4个或更多题目且包含所有必要信息：1分 提交了2个或以上题目但不足4个：0.5分 少于：0分	1 分
清晰头像、PDF文件、MD/DOC附件	包含清晰的Canvas头像、PDF文件以及MD或DOC格式的附件：1分 缺少上述三项中的任意一项：0.5分 缺失两项或以上：0分	1 分
学习总结和个人收获	提交了学习总结和个人收获：1分 未提交学习总结或内容不详：0分	1 分
总得分： 5	总分满分：5分	

说明：

1. 解题与记录：

对于每一个题目，请提供其解题思路（可选），并附上使用Python或C++编写的源代码（确保已在OpenJudge， Codeforces， LeetCode等平台上获得Accepted）。请将这些信息连同显示“Accepted”的截图一起填写到下方的作业模板中。（推荐使用Typora <https://typoraio.cn> 进行编辑，当然你也可以选择Word。）无论题目是否已通过，请标明每个题目大致花费的时间。

2. 课程平台：

课程网站位于Canvas平台（<https://pku.instructure.com>）。该平台将在第2周选课结束后正式启用。在平台启用前，请先完成作业并将作业妥善保存。待Canvas平台

激活后，再上传你的作业。

3. **提交安排：**提交时，请首先上传PDF格式的文件，并将.md或.doc格式的文件作为附件上传至右侧的“作业评论”区。确保你的Canvas账户有一个清晰可见的本人头像，提交的文件为PDF格式，并且“作业评论”区包含上传的.md或.doc附件。
4. **延迟提交：**如果你预计无法在截止日期前提交作业，请提前告知具体原因。这有助于我们了解情况并可能为你提供适当的延期或其他帮助。

请按照上述指导认真准备和提交作业，以保证顺利完成课程要求。

1. 题目

E868.二进制间距

bit manipulation, <https://leetcode.cn/problems/binary-gap/>

主要是练习面向对象编程写法，这样力扣题目，笔试都没有问题了。机考时候，不是必须OOP，能AC就可以。

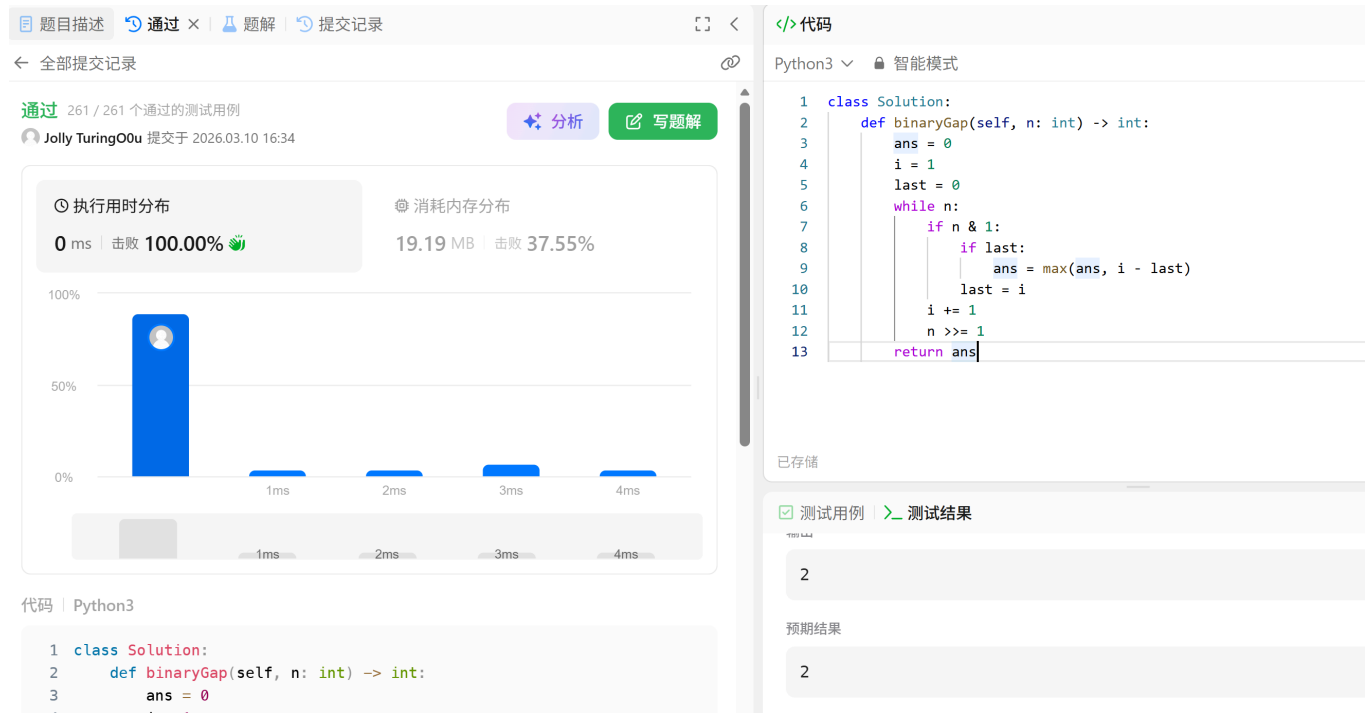
思路：

用 `last` 记录上一次的 1 出现的位置，若为0则还未出现过，把 `n` 按二进制位展开判断是否为 1

代码：

```
class Solution:
    def binaryGap(self, n: int) -> int:
        ans = 0
        i = 1
        last = 0
        while n:
            if n & 1:
                if last:
                    ans = max(ans, i - last)
                last = i
            i += 1
            n >>= 1
        return ans
```

代码运行截图（至少包含有"Accepted"）



M304.二维区域和检索 - 矩阵不可变

prefix sum, <https://leetcode.cn/problems/range-sum-query-2d-immutable/>

思路：

二维前缀和，presum 数组存储左上角的元素和

代码：

```
class NumMatrix:

    def __init__(self, matrix: List[List[int]]):
        m = len(matrix)
        n = len(matrix[0])
        presum = [[0] * (n + 1) for _ in range(m + 1)]
        for i in range(1, m + 1):
            for j in range(1, n + 1):
                presum[i][j] = (
                    presum[i - 1][j]
                    + presum[i][j - 1]
                    - presum[i - 1][j - 1]
                    + matrix[i - 1][j - 1]
                )
            self.presum = presum

    def sumRegion(self, row1: int, col1: int, row2: int, col2: int) -> int:
```

```

presum = self.presum
return (
    presum[row2 + 1][col2 + 1]
    - presum[row1][col2 + 1]
    - presum[row2 + 1][col1]
    + presum[row1][col1]
)

```

Your NumMatrix object will be instantiated and called as such:

```

# obj = NumMatrix(matrix)
# param_1 = obj.sumRegion(row1,col1,row2,col2)

```

代码运行截图（至少包含有"Accepted"）

通过 22 / 22 个通过的测试用例

Jolly Turing00u 提交于 2026.03.10 16:53

分析

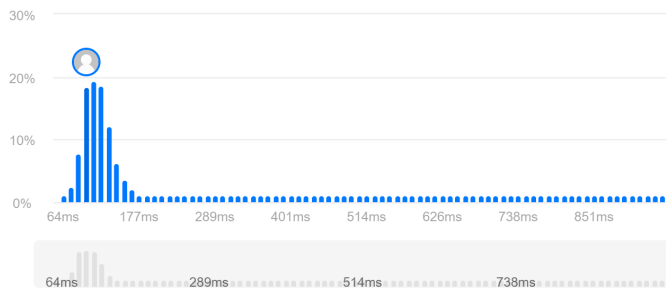
写题解

执行用时分布

102 ms | 击败 75.21%

消耗内存分布

32.78 MB | 击败 63.18%



代码 | Python3

```

1 class NumMatrix:
2
3     def __init__(self, matrix: List[List[int]]):
4         m = len(matrix)
5         n = len(matrix[0])

```

```

1 class NumMatrix:
2
3     def __init__(self, matrix: List[List[int]]):
4         m = len(matrix)
5         n = len(matrix[0])
6         presum = [[0] * (n + 1) for _ in range(m + 1)]
7         for i in range(1, m + 1):
8             for j in range(1, n + 1):
9                 presum[i][j] = (
10                     presum[i - 1][j]
11                     + presum[i][j - 1]
12                     - presum[i - 1][j - 1]
13                     + matrix[i - 1][j - 1]
14                 )
15         self.presum = presum

```

已存储

测试用例 > 测试结果

通过 执行用时: 0 ms

Case 1

输入

["NumMatrix", "sumRegion", "sumRegion", "sumRegion"]

M1680.连接连续二进制数字

bit manipulation, <https://leetcode.cn/problems/concatenation-of-consecutive-binary-numbers/>

思路:

按照题意实现拼接

代码:

```

class Solution:
    def concatenatedBinary(self, n: int) -> int:
        mod = 10 ** 9 + 7
        ans = 0
        for i in range(1, n + 1):

```

```
ans = (ans << i.bit_length() | i) % mod
return ans
```

代码运行截图 (至少包含有"Accepted")

 代码  通过 ×



← 全部提交记录

通过 403 / 403 个通过的测试用例

 Jolly Turing00u 提交于 2026.03.10 11:13

 分析

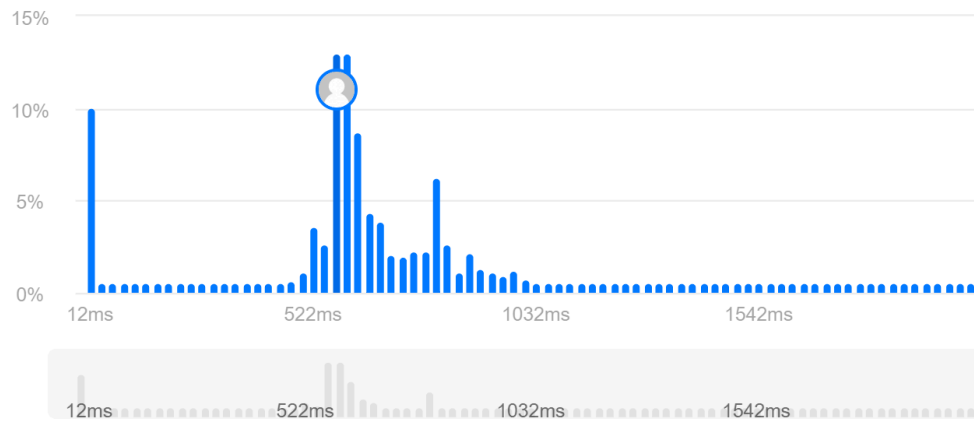
 写题解

🕒 执行用时分布

587 ms | 击败 68.04% 🏆

💾 消耗内存分布

19.06 MB | 击败 69.92% 🏆



代码 | Python3

```
1 class Solution:
2     def concatenatedBinary(self, n: int) -> int:
3         mod = 10 ** 9 + 7
```

M1461.检查一个字符串是否包含所有长度为 K 的二进制子串

bit manipulation, <https://leetcode.cn/problems/check-if-a-string-contains-all-binary-codes-of-size-k/>

思路:

用 flag 数组当作哈希表, 表示每个长度为 k 的二进制数是否被表示, 用滑动窗口的方法遍历 s 字符串所有长度为 k 的子串, 记入flag

代码:

```
class Solution:
```

```
def hasAllCodes(self, s: str, k: int) -> bool:
    n = len(s)
    if n < 1 << k: return False
    x = int(s[:k-1], 2) if k > 1 else 0
    m = (1 << k) - 1
    flag = bytearray(1 << k)
    count = 0
    for i in range(k - 1, n):
        x = x << 1 & m | (1 if s[i] == '1' else 0)
        if not flag[x]:
            flag[x] = 1
            count += 1
            if count == 1 << k:
                return True
    return False
```

代码运行截图（至少包含有"Accepted"）

← 全部提交记录

通过 201 / 201 个通过的测试用例

Jolly Turing00u 提交于 2026.03.10 11:36

✦ 分析

✎ 写题解

⌚ 执行用时分布

115 ms | 击败 98.77% 🏆

💾 消耗内存分布

21.39 MB | 击败 98.23% 🏆



代码 | Python3

```
1 class Solution:
2     def hasAllCodes(self, s: str, k: int) -> bool:
3         n = len(s)
```

M30178:数字华容道（Easy Version）

merge sort, binary indexed tree, <http://cs101.openjudge.cn/practice/30178/>

思路：

将数字按行展开，考虑 0 块上下时是将数字向前/向后移动 $n-1$ 格这样的性质，转化为判定序列的逆序对数的奇偶性

代码：

```
import sys
import array

class BIT:
    def __init__(self, n):
        self.n = n
        self.tree = bytearray(n + 1)

    def add(self, i: int, x: int) -> None:
        while i <= self.n:
            self.tree[i] ^= x
            i += i & -i

    def query(self, i: int) -> int:
        ans = 0
        while i:
            ans ^= self.tree[i]
            i &= i - 1 # i -= i & -i
        return ans

def main():
    it = iter(sys.stdin.read().split())
    t = int(next(it))
    for _ in range(t):
        n = int(next(it))
        m = n * n - 1

        bit = BIT(m)
        ans = tmp = 0
        index = 1
        for i in range(n):
            for j in range(n):
                x = m + 1 - int(next(it))
                if x <= m:
                    ans ^= bit.query(x - 1) & 1
                    bit.add(x, 1)
                    index += 1
                else:
                    tmp = n - 1 - i

            if not n & 1:
```

```
        ans ^= tmp & 1
    print('no' if ans & 1 else 'yes')

if __name__ == '__main__':
    main()
```

代码运行截图 (至少包含有"Accepted")

#51942627提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: Accepted

源代码

```
import sys

class BIT:
    def __init__(self, n):
        self.n = n
        self.tree = [0] * (n + 1)

    def add(self, i: int, x: int) -> None:
        while i <= self.n:
            self.tree[i] ^= x
            i += i & -i

    def query(self, i: int) -> int:
        ans = 0
        while i:
            ans ^= self.tree[i]
            i &= i - 1 # i -= i & -i
        return ans
```

基本信息

#: 51942627
题目: 30178
提交人: 25n2500011619
内存: 131148kB
时间: 4640ms
语言: Python3
提交时间: 2026-03-11 21:00:04

T30201: 旅行售货商问题

bitmask dp, <http://cs101.openjudge.cn/practice/30201/>

思路:

dp[i][j] 的定义是: i: 经过了哪些城市 (状态/集合), j: 目前停留在哪个城市。从城市0 出发遍历其他城市。

代码:

```
def main():
    n = int(input())
    cost = [list(map(int, input().split())) for _ in range(n)]
    size = 1 << n
    inf = float('inf')
    dp = [[inf] * n for _ in range(size)]
    dp[1][0] = 0
    for mask in range(1, size, 2):
        for i in range(mask.bit_length()):
```



```

        if mask & (1 << i):
            for j in range(1, n):
                if mask & (1 << j):
                    continue
                tmp = dp[mask][i] + cost[i][j]
                nxt = mask | (1 << j)
                if dp[nxt][j] > tmp:
                    dp[nxt][j] = tmp
            print(min(dp[size - 1][i] + cost[0][i] for i in range(1, n)))

if __name__ == '__main__':
    main()

```

代码运行截图 (至少包含有"Accepted")

#51973307提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: Accepted

源代码

```

def main():
    n = int(input())
    cost = [list(map(int, input().split())) for _ in range(n)]
    size = 1 << n
    inf = float('inf')
    dp = [[inf] * n for _ in range(size)]
    dp[1][0] = 0
    for mask in range(1, size, 2):
        for i in range(mask.bit_length()):
            if mask & (1 << i):
                for j in range(1, n):
                    if mask & (1 << j):
                        continue
                    tmp = dp[mask][i] + cost[i][j]
                    nxt = mask | (1 << j)
                    if dp[nxt][j] > tmp:
                        dp[nxt][j] = tmp

```

基本信息

#: 51973307
 题目: 30201
 提交人: 25n2500011619
 内存: 102432kB
 时间: 740ms
 语言: PyPy3
 提交时间: 2026-03-14 16:09:30

2. 学习总结和个人收获

如果发现作业题目相对简单，有否寻找额外的练习题目，如“数算2025spring每日选做”、LeetCode、Codeforces、洛谷等网站上的题目。

这周主要是深入学习了树状数组，之前是知道加减 lowbit 的做法但是不知道原理，学习了灵神的讲解以后感觉变清晰了很多。树状数组将每个前缀区间分解成右端点不重复的关键区间，且每个位置被覆盖的关键区间数和每个前缀区间覆盖的关键区间数都是 $O(\log n)$ 量级的，可以快速处理单点加、区间查询或区间加、单点查询（通过差分与前者相同），可以用于处理逆序对数目，且代码短小，是非常优美的数据结构。

重新复习了上学期的状压DP，这次学了二进制操作以后感觉更清晰了。

完成了数字华容道的easy version，但是hard version还没有研究。